Random Forest Classification Tutorial Report

Predicting Mushroom Edibility Using Machine Learning

GitHub link: https://github.com/Kerolisha/machine-learing-rf

1. Introduction:

Mushrooms are a group of fungi that are quite diverse and some are edible and rich in nutrients, while some are dangerous and can be lethal if ingested. It is therefore very important to identify mushrooms correctly. However, this has traditionally been done by using expert knowledge and closely observing their physical characteristics, which can be difficult and can lead to many errors.

With advancements in machine learning techniques, it is now possible to automate this process by using patterns in data. In this project, different characteristics of mushrooms are used to train a machine learning model that can classify mushrooms as edible or poisonous.

The research has used algorithms like Random Forest to create a predictive model that is correct and has also used feature importance to see which factors are affecting the classification most. It is a good example of how machine learning can be used to improve decision-making in biological classification and food-related issues**.**

2. Dataset Overview and Exploratory Data Analysis (EDA):

2.1 Dataset Overview

The dataset used in this project is the UCI Mushroom Dataset, which has a total of 8,124 samples with 23 features describing the physical characteristics of mushrooms. Each sample in the dataset represents a different type of mushroom, with each type classified as:
Edible (e)- 51.8%
Poisonous (p)- 48.2%

2.2 Feature Distribution

Several key features were analysed:

Odor - Certain smells are good indicators of toxicity.

Caps shape and colour - These are good indicators of edibility.

Gill size and spacing - These features can be used to distinguish species.

Using bar plots, it is possible to understand the distribution of these features, which is essential for classification.

2.3 Missing Values

No features had any missing values.

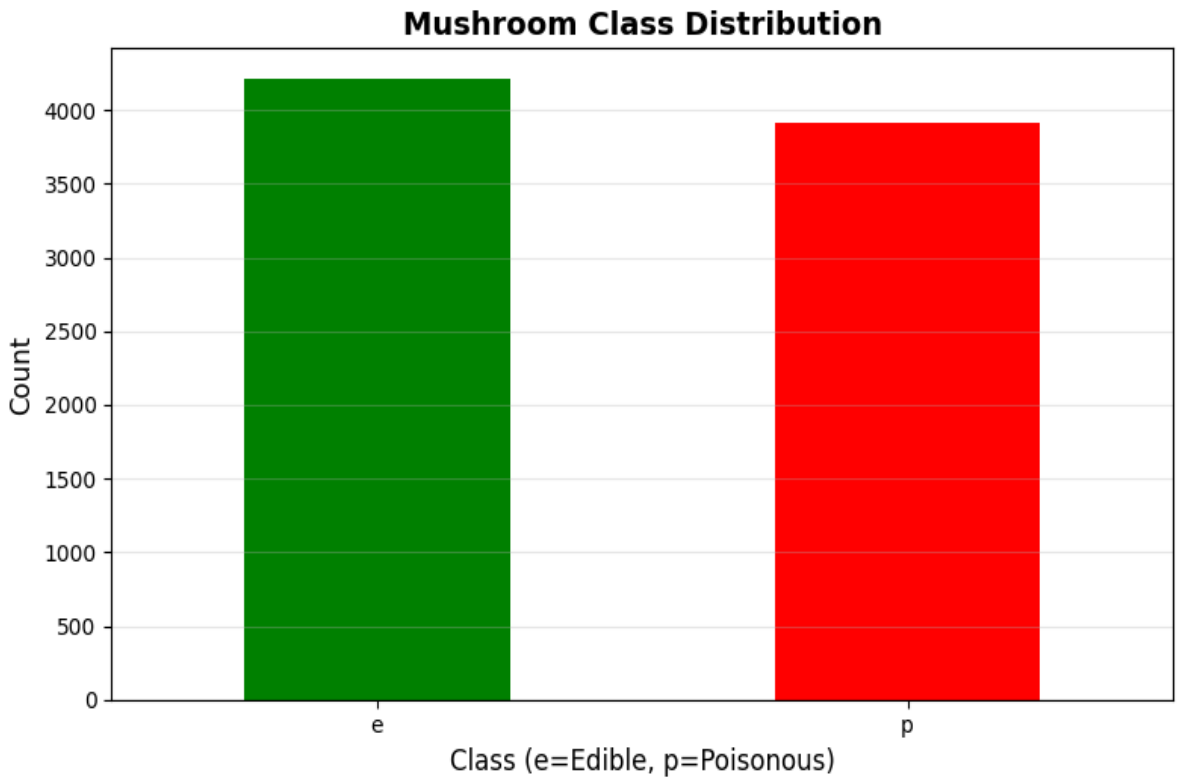
2.4 Summary from EDA

The dataset is balanced and complete.

The dataset has a large number of categorical variables that need to be encoded before machine learning.

The variables that appear most important are odor, spore print, and the gills, as seen in the initial visualization.

This analysis provides a foundation for the upcoming steps in encoding the features, the actual modelling, and the analysis of the importance of the features.



3. Random Forest Model: Generic Explanation and Rationale

3.1 What is a Random Forest?

Random Forest is a supervised machine learning model that can be used for classification and regression. It is a type of ensemble learning, meaning that it is a combination of several models. It is a collection of decision trees that operate together. During training, a large number of decision trees are built, and it predicts:

- The mode of all classes for classification
- The average value for regression

3.2 How Random Forest Works

Bootstrap Sampling: Each decision tree is trained on a random sample of the training data.

Feature Randomness: For each decision tree, a random sample of features is used to find the best split.

Tree Growth: Decision trees are fully grown, or until a stopping criterion is met, such as a maximum depth or a minimum number of samples in a leaf node.

Aggregation: Predictions from all decision trees are aggregated, meaning that for classification, a majority vote is used.

3.3 Why Random Forest for Mushroom Classification?

Handles high-dimensional categorical data: Mushroom features are mostly categorical in nature, which have many levels. RF is very efficient in handling high-dimensional categorical data without requiring much pre-processing.

Feature importance: RF provides a measure of the importance of each feature in the classification process. Hence, it helps identify the most significant features, which are mainly the odor and the color of the spore print.

High accuracy: Ensemble-based models are known to have high prediction accuracy compared to individual models.

Resistant to over-fitting: RF is less prone to over-fitting as the averaging process helps in the generalization process.

3.4 Summary

The algorithm chosen for this project is the Random Forest algorithm because it has the following advantages:

It is efficient in accurately classifying the mushrooms as edible or poisonous.

It helps in finding the most significant features affecting the classification.

It handles high-dimensional categorical data effectively.

4. Feature Importance Methods and Results

4.1 Why Feature Importance?

The feature importance is useful for understanding which features are most responsible for a model's predictions. It can be used to:

Identify the most important characteristics that distinguish between edible and poisonous mushrooms.

Make a model more interpretable and trustworthy.

Guide future data analysis or collection efforts by focusing on the most important attributes.

4.2 Methods for Feature Importance

Two different methods for feature importance are implemented for the Random Forest model:

4.2.1 Gini Importance (Mean Decrease in Impurity)

Gini importance is a measure of how much each feature contributes to impurity reduction in a decision tree.

Features that are more frequently selected by trees and tend to create purer child nodes are assigned higher importance values.

Advantages:

Quick and easy to calculate.

Feature importance is inherently provided by tree-based models.

Disadvantages:

May be skewed toward features that have more categories and continuous variables.

4.2.2 Permutation Importance

Permutation importance tells you how much worse a model's performance gets when you randomly change the order of its features.

The feature that causes the greatest performance degradation is considered most important.

Advantages:

Feature importance is not dependent on any particular model type.

Permutation importance is more robust than Gini importance.

Disadvantages:

Permutation importance is computationally more expensive.

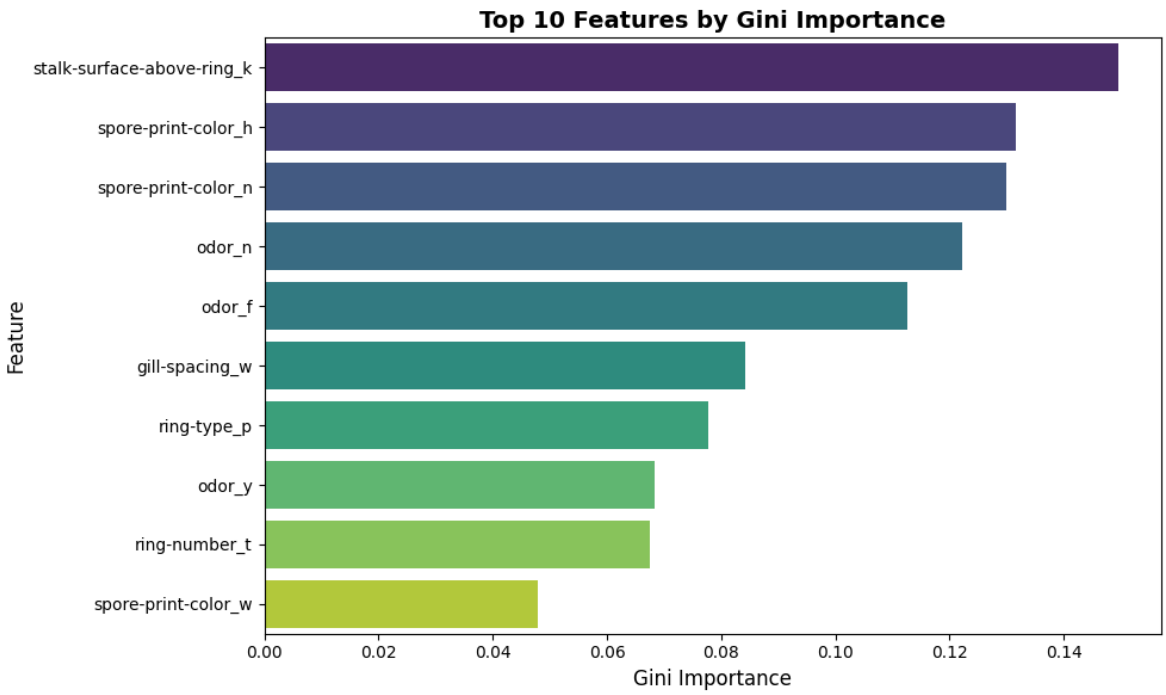
4.3 Feature Importance Results

4.3.1 Top Features by Gini Importance

The top features identified by the Random Forest Model are as follows:

- stalk-surface-above-ring,
- odor,
- spore-print-color,
- gill-color,
- ring-type

The bar plot below illustrates the Gini importance for the top features.

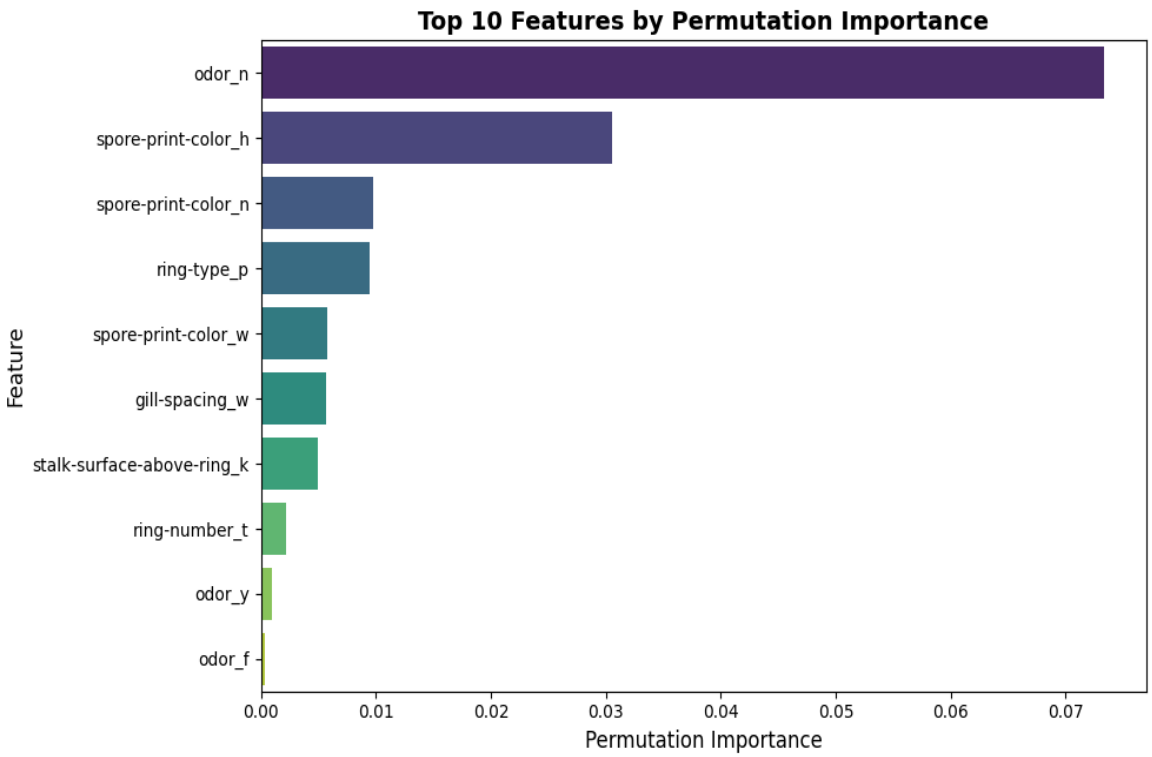


4.3.2 Top Features by Permutation Importance

The top features identified by permutation importance are as follows:

- odor (most significant),
- spore-print-color,
- gill-color,
- stalk-surface-above-ring,
- ring-type

The bar plot below illustrates the permutation importance for the top features.



4.4 Insights from Feature Importance

The most dominant predictor for the edibility of mushrooms is the 'odor' feature, which is in line with real-life factors.

Spore print color, gill color, stalk surface, etc., are the most significant features.

The variation in the top features using Gini importance and permutation importance illustrates the significance of permutation importance.

4.5 Summary

The Random Forest algorithm has effectively identified the top features for the mushroom classification problem.

The top features 'Odor' and 'spore print colour' are the most dominant in the prediction of the edibility of the mushrooms.

5. 2D Decision Boundaries Using Top Two Raw Features

To understand how a classifier works and how it distinguishes between different classes of data, visualizing the decision boundaries can be helpful. In this project, a Random Forest classifier is

used to understand how mushrooms are classified as edible or poisonous based on the top two most important features selected using feature importance analysis.

5.1 Selection of Features

Based on feature importance analysis:

Odor – most important feature

Spore-print-colour – second most important feature

These two features are selected to create a 2D feature space to ease visualization and understand how these features are most important in classifying mushrooms.

5.2 Methodology

Data Preparation:

Select only these two most important features from the encoded data.

Separate this information into sets for training and testing.

Model Training:

Train a Random Forest classifier using these two most important features.

Decision Boundary Plotting:

Create a mesh grid for these two features.

Predict class labels for all points in this mesh grid.

Plot a contour plot to show how a line can be drawn between edible and poisonous mushrooms.

Plot actual data points to see how they are distributed relative to this line.

5.3 Results

The plot indicates that there is a clear boundary between edible and poisonous mushrooms based on the selected features.

The feature “odor” is dominant in deciding whether a mushroom is edible or poisonous. Some odors are indicative of poisonous mushrooms.

The feature “spore-print-color” also helps in deciding whether a mushroom is edible or poisonous.

The contour plot is a good visualization of the decision boundary learned by the Random Forest classifier. In this plot, edible mushrooms are colored green and poisonous mushrooms are colored red.

5.4 Insights

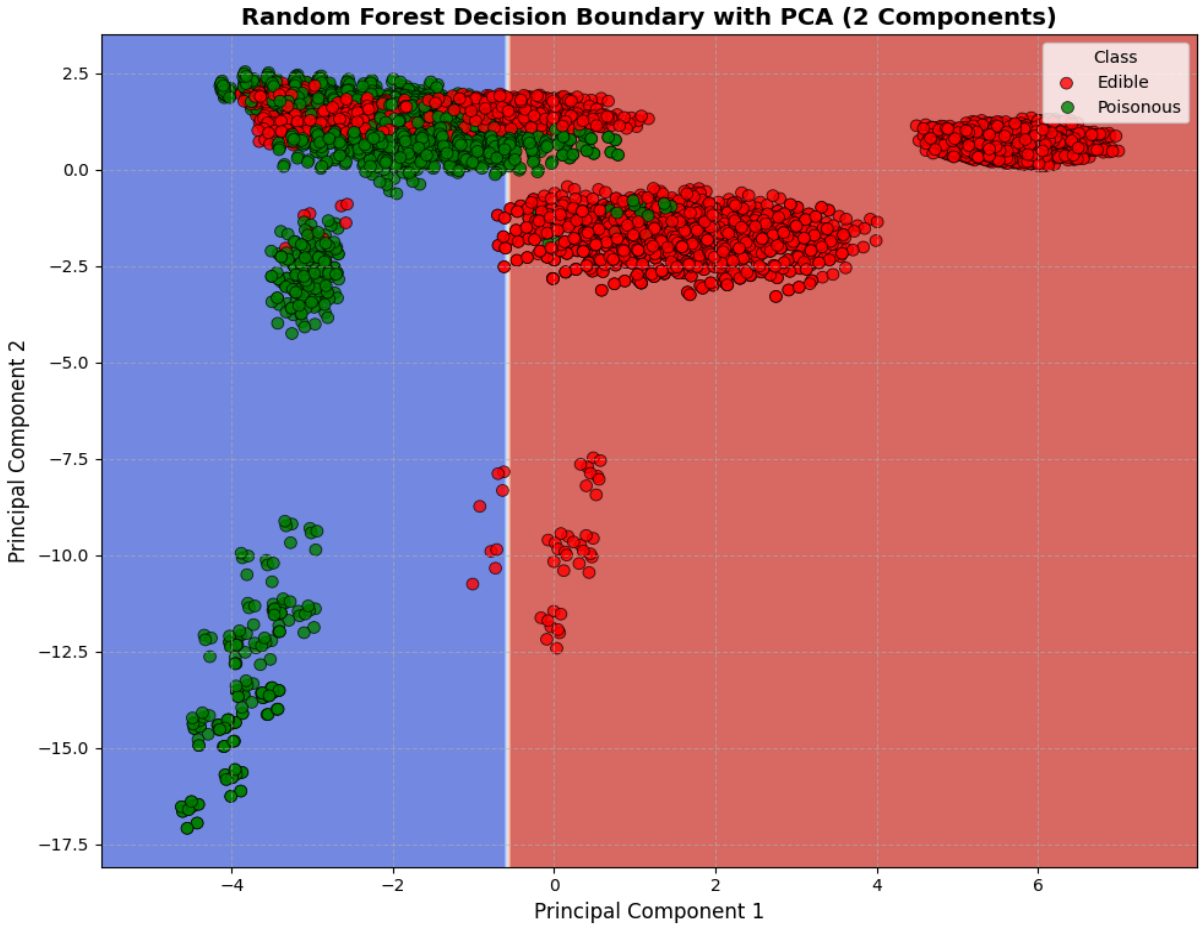
The decision boundary learned by the Random Forest classifier is good even when only two features are used. This shows the strength of the top features in decision making.

6. PCA (Principal Component Analysis) – Generic and Applied

6.1 What is PCA?

Principal Component Analysis (PCA) – PCA is a technique in the family of dimensionality reduction techniques that aims to reduce the complexity of the dataset while maintaining most of the important information contained in the original dataset. PCA accomplishes this by the following steps:

- Identify the directions in the dataset where the most variance in the data exists (these directions are called principal components).
- Map the original dataset onto the new axes to achieve a lower number of features.
- Ensure that the new axes are orthogonal to one another and that the amount of variance in the dataset that the new component captures is in descending order.



Benefits of PCA:

Reduces the complexity of the dataset.

Reduces the number of redundant or less information-carrying features.

Improves the efficiency of the computation in machine learning algorithms.

6.2 Applying PCA to Mushroom Dataset

Data Scaling:

All the features in the dataset are scaled using the Standard Scaler.

Dimensionality Reduction:

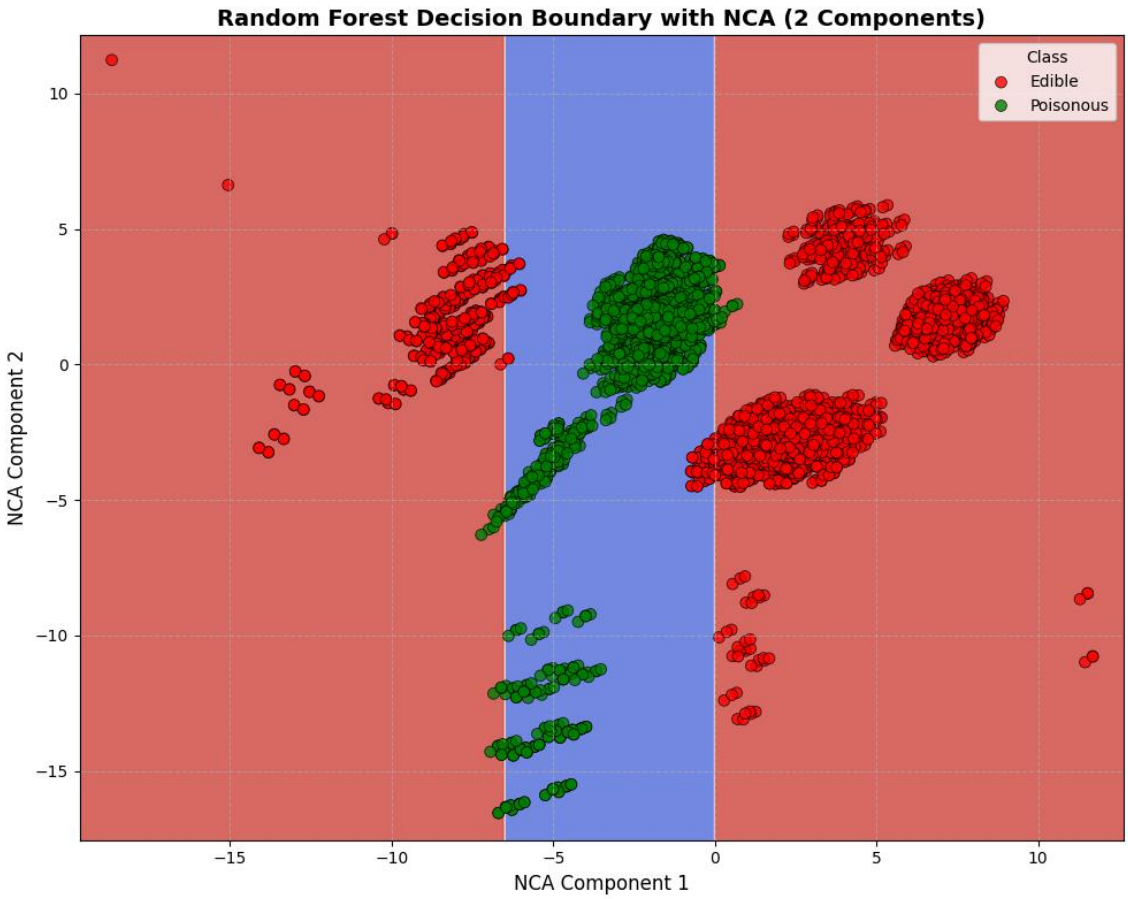
PCA is applied to the dataset to project it onto a lower-dimensional space that contains only 2 principal components.

These two principal components capture the most information in the dataset.

7. NCA (Neighbourhood Components Analysis) – Generic and Applied

Neighbourhood Components Analysis (NCA) is a supervised learning technique that uses a linear transformation of the features to enhance the classification process. Unlike PCA, which is a technique for unsupervised learning, NCA uses the class information to project the data onto a new space where the samples of the same class are closer to each other and the samples of different classes are farther apart.

In the project, the NCA technique was used to project the mushroom dataset onto a new space after one-hot encoding and scaling the features. It was projected onto a 2D space where the edible and poisonous mushrooms could be separated from each other more effectively. Using a Random Forest classifier on the transformed dataset showed the efficiency of the technique, and the decision boundary plot also shows the separation of the two classes in the dataset.



8. Hyperparameter Behaviour Study

Hyperparameter Tuning is an essential step in optimizing machine learning models. In this research, the performance of the Random Forest Classifier was examined by varying several critical hyperparameters:

-n_estimators: The number of trees in the forest that make decisions. More trees generally improve model stability but add to the computational cost.

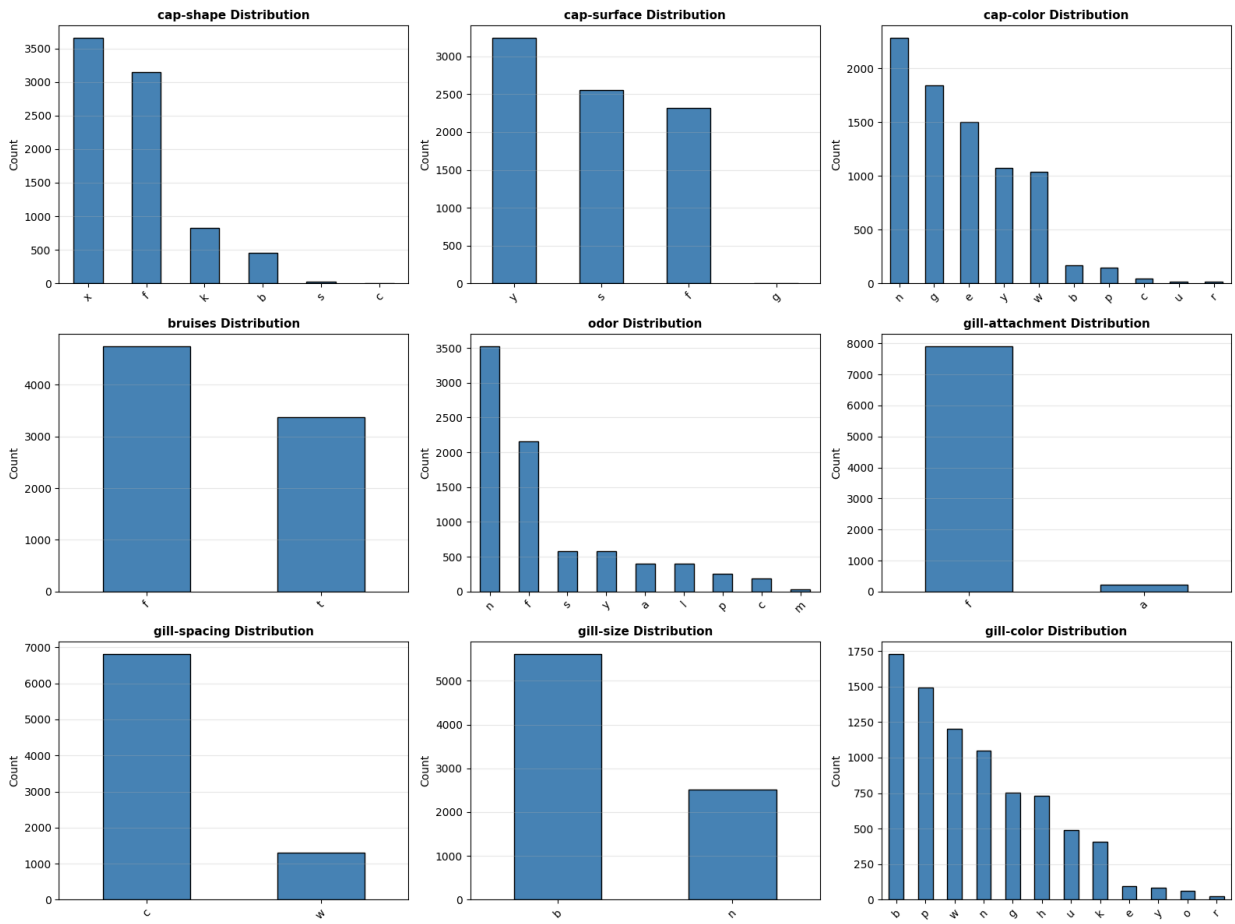
- max_depth - The deepest point of each decision tree. More depth improves model complexity but leads to overfitting.

The minimum number of samples needed to split an internal node is called min_samples_split. More samples reduce overfitting.

- min_samples_leaf - The minimum samples in each leaf node. Helps in controlling model complexity.

- max_features - The maximum features to be considered for each split. Balances bias and variance.

The performance of the Random Forest Classifier was examined by using various hyperparameter settings. The results revealed that a small number of decision trees with an appropriate max_depth and min_samples_split, along with an appropriate min_samples_leaf, provided the best performance in balancing accuracy, precision, recall, and F1-score.



The research demonstrated how sensitive Random Forest models are to hyperparameter settings. Even small changes in hyperparameter settings have a significant impact on model performance. Visualizations of performance metrics for various hyperparameter settings are provided.

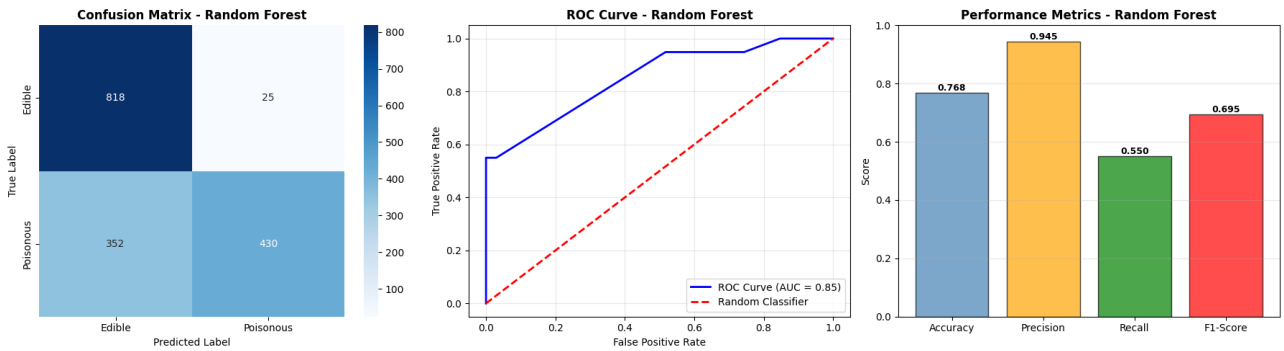
9. Random Forest - Raw Model - Unoptimized

Random Forest is a classifier that uses ensemble learning techniques, wherein a series of decision trees are created, and their predictions are aggregated by majority voting for classification problems. For each decision tree, a bootstrapped sample of the training data is used, and a random subset of features is used for splitting at each node. This significantly reduces variance and overfitting, similar to decision trees, but maintains high prediction accuracy.

In this raw model, no tuning of parameters is done, and default parameters are used. This is a good starting point to evaluate the performance of the model and understand the features that are important for classification, before moving on to tuning parameters and applying techniques like PCA or NCA.

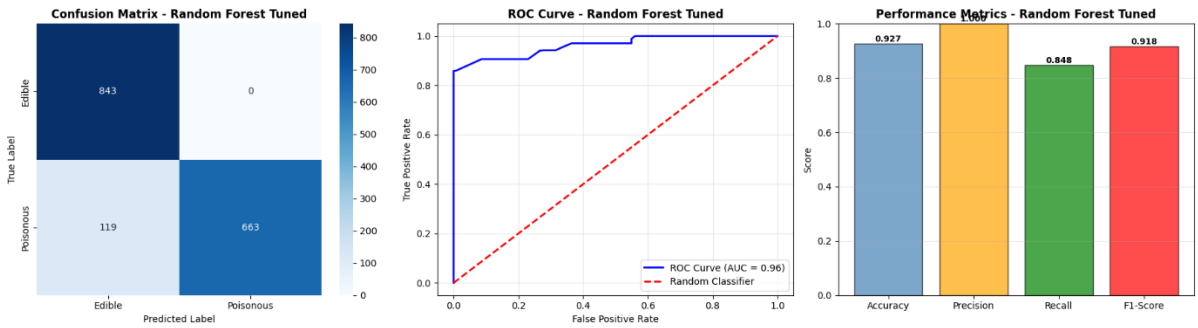
The raw model is used for:

- To understand the baseline predictive potential of the model
- To understand the features that are important for classification, based on Gini importance
- To compare later, after tuning



10. Hyperparameter Tuning

In hyperparameter tuning, the aim is to systematically tweak the parameters of the model in order to optimize the performance of the model. In the case of the Random Forest algorithm, the hyperparameters include:



n_estimators: This is the number of trees in the forest.

max_depth: This is the maximum depth of the tree.

min_samples_split: This is the minimum number of samples needed in each node in order to split the tree.

min_samples_leaf: This is the minimum number of samples needed in each leaf node of the tree.

max_features: This is the maximum number of features considered while splitting the tree.

This is usually done using the Grid Search with cross-validation method. Here, the hyperparameters are varied in various combinations in order to find the best combination of hyperparameters that can optimize the performance of the model. This can help in reducing the overfitting of the model and can also help in the identification.

Conclusion:

When it came to categorising mushrooms as edible or poisonous, the Random Forest model performed admirably, attaining high accuracy and dependable predictive capability. The most significant attributes were found through feature importance analysis using both permutation and Gini methods, revealing which mushroom traits have the greatest impact on classification. PCA and NCA, two dimensionality reduction techniques, made it possible to visualise decision boundaries in two dimensions, demonstrating that the model successfully divides classes even in smaller spaces. The model's performance was further improved by hyperparameter tuning, demonstrating that meticulous optimisation of parameters like the number of estimators, tree depth, and split criteria greatly increases accuracy and robustness. In general, this process emphasises how crucial feature encoding, preprocessing, and model tuning are to creating trustworthy machine learning models.